UNIVERSITI TEKNOLOGI MARA
MARKING SCHEME

COURSE	:	COMPUTER ORGANIZATION AND ARCHITECTURE
COURSE CODE	:	CSC429
EXAMINATION	:	NOVEMBER 2026 (MOCK)
TIME	:	3 HOURS

MARKING SCHEME NOTES

1. This marking scheme accompanies the CSC429 mock paper (November 2026). Generated answers show working. Real-bank questions carry a labelled unofficial AI model answer, not an official scheme.

2. Marking points show the answer and mark allocation. Check model-generated real-bank points against the source paper.

3. Part A = 4 x 5 + 6 x 10; Part B = 1 x 20; total = 100 marks.

4. Generated slots: 2. Real-bank slots: 9. Unofficial model answers:

   9/9.
   For local study use only; model answers are unofficial.

5. Structure from 20 unique CSC429 past papers; every real question remains traceable to its source page.

FOR MARKERS' USE ONLY

This marking scheme consists of 10 printed pages

© Hak Cipta Universiti Teknologi MARA

PART A — MARKING GUIDE

QUESTION A1 (5 marks)

Topic: Computer generations and history

Source: CSC429 (4).pdf, page(s) 2

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Fourth generation. The production of personal computers is the main feature of the fourth generation (microprocessors), which also brought the lowest prices, fast processing and low power consumption.

(b) Fifth generation. Machines that emulate human intelligence and process/respond to natural language are characteristic of the fifth generation (artificial intelligence).

(c) Third generation. In the third generation (integrated circuits), reduced heat meant air conditioning was no longer required, though a fan was still needed for heat discharge.

(d) Third generation. The integrated circuit, comprising transistors, resistors and capacitors, was invented by Jack Kilby in the third generation.

(e) Second generation. Assembly language was introduced in the second generation to replace machine language for more reliable and improved performance.

Unofficial marking guide:

- *1 mark: (a) Fourth generation*
- *1 mark: (b) Fifth generation*
- *1 mark: (c) Third generation*
- *1 mark: (d) Third generation*
- *1 mark: (e) Second generation*

Model confidence: high.

Model note: Assumes standard computer-generation classification: first (vacuum tubes), second (transistors), third (integrated circuits), fourth (microprocessors), fifth (AI). No figure/table required.

QUESTION A2 (5 marks)

Topic: Instruction formats and addressing modes

Source: CSC429 (1).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The addressing technique illustrated in Figure 2 is direct addressing (also called absolute addressing). The instruction contains an opcode and an address field A, and the memory operand is stored at memory address A; the address field directly gives the effective address of the operand.

(b) In direct addressing, the address field A of the instruction is placed directly on the memory address bus. The CPU reads the operand from memory location A (or writes to it) without any further address calculation. The effective address is EA = A. For example, if the instruction is LOAD A and A = 2000, the processor fetches the contents of memory location 2000 and loads it into the accumulator.

(c) Advantage: Direct addressing is simple and fast because no additional address calculation or extra memory access is needed to determine the effective address; the operand address is available immediately in the instruction. Disadvantage: The range of addresses is limited by the number of bits in the address field, so only a restricted portion of memory can be addressed directly; also, the address field increases instruction length.

Unofficial marking guide:

- *1 mark: (a) Correctly labels the technique as direct addressing (accept absolute addressing).*
- *2 marks: (b) Explains that the address field A is used directly as the effective address (1 mark); explains that the operand is fetched from memory location A without further calculation (1 mark).*
- *2 marks: (c) One valid advantage (1 mark) and one valid disadvantage (1 mark).*

Model confidence: high.

Model note: Figure 2 is assumed to show an instruction with an opcode and address field A, with the operand stored at memory address A, consistent with the extracted text. If the figure shows an additional level of indirection, the answer would be indirect addressing; however, the text supports direct addressing.

QUESTION A3 (5 marks)

Topic: Data representation and number systems

Answer: 277410 = 1010110101102

• 1 mark for identifying the target base.
• 1 mark for a correct repeated-division or place-value method.
• 1 mark for the correct digit sequence.
• 1 mark for the correct base subscript.
• 1 mark for a complete, consistent workflow.

QUESTION A4 (5 marks)

Topic: Parallel and multiprocessor architecture

Source: CSC429 (5).pdf, page(s) 6

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

Taxonomy of parallel processor architecture (with examples)

A standard way to classify parallel processors is Flynn's taxonomy, which sorts architectures by the number of instruction streams and data streams they process simultaneously. The four categories, each with an example, are:

- SISD (Single Instruction, Single Data): one instruction stream operates on one data stream per cycle. This is the conventional uniprocessor (e.g., a classic single-core von Neumann CPU such as an Intel 8086 or a basic scalar processor).
- SIMD (Single Instruction, Multiple Data): one control unit issues a single instruction that is applied in parallel to many data elements by multiple processing elements. Examples include array/vector processors (e.g., CRAY-1 vector unit) and GPU/SIMD extensions such as MMX/SSE, or an array processor like the ILLIAC IV.
- MISD (Multiple Instruction, Single Data): several instruction streams act on the same data stream; this is rare and mainly theoretical, exemplified by systolic arrays or fault-tolerant replicated pipelines.
- MIMD (Multiple Instruction, Multiple Data): multiple processors each execute independent instruction streams on independent data. Examples are multicore processors and multiprocessor systems (e.g., a shared-memory SMP server or a cluster of workstations).

(If the figure uses the alternative structure of shared-memory vs. distributed-memory MIMD, the examples are: shared-memory multiprocessor/SMP for tightly coupled systems, and message-passing multicomputer/cluster for loosely coupled systems.)

Example summary table:

Category	Instruction streams	Data streams	Example
SISD	1	1	Single-core scalar CPU
SIMD	1	Many	Vector/array processor, GPU
MISD	Many	1	Systolic array
MIMD	Many	Many	Multicore / SMP / cluster

Unofficial marking guide:

• 1 mark: correctly naming the four Flynn categories (SISD, SIMD, MISD, MIMD) or the figure's equivalent taxonomy levels
• 2 marks: correct definition/description of each category in terms of instruction and data streams (0.5 mark per category)
• 2 marks: a valid example given for each category (0.5 mark per category)
Model confidence: medium.
Model note: The figure (CSC429-2016-06-D1) was not available as an image, only its presence is known. The answer assumes the taxonomy is Flynn's classification, the most common parallel-processor taxonomy in CSC429. If the figure instead shows shared-memory vs distributed-memory (tightly/loosely coupled) MIMD, the examples in the parenthetical note should be used and marks reallocated accordingly.

QUESTION A5 (10 marks)

Topic: Input/output and interrupts

Source: CSC429 (16).pdf, page(s) 4

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Two methods to handle multiple interrupts: (1) Software polling / priority polling: When multiple interrupts occur, the processor branches to a common interrupt service routine that polls each device in priority order; the highest-priority requesting device is serviced first. This is flexible because priority can

© Hak Cipta Universiti Teknologi MARA

be changed in software, but it is slower because polling takes time. (2) Daisy chaining (hardware priority): Interrupt request lines are connected in a chain; the interrupt acknowledge signal propagates through devices in a fixed hardware priority order. The first device in the chain that has requested an interrupt is serviced first. This is fast and requires little software overhead, but priority is fixed by hardware and a faulty device can break the chain. (Other acceptable methods: vectored interrupts, multiple interrupt lines, or priority encoder.)

(b) One example of a non-maskable interrupt (NMI): a power failure or power-supply voltage drop (also acceptable: memory parity error, bus error, or a hardware reset button). Such an interrupt cannot be disabled by software and is used for critical events.

(c) DMA is faster than interrupt-driven and programmed I/O because the DMA controller transfers data directly between the I/O device and memory without the CPU executing instructions for each byte/word. The CPU only initializes the DMA (sets address, count, direction) and is interrupted once at the end of the block transfer. In programmed I/O the CPU continuously polls and moves each data item, and in interrupt-driven I/O the CPU is interrupted for every data item and executes an ISR, so both incur high CPU overhead and many bus cycles. DMA therefore achieves higher throughput and frees the CPU for other work.

(d) A 'hot connection' means the USB device can be connected or disconnected while the computer is powered on and running, without shutting down or rebooting the system. The interface is designed to be hot-swappable and supports plug-and-play detection and configuration of the device.

Unofficial marking guide:

• 1 mark: correctly naming the first method to handle multiple interrupts (e.g., software polling or daisy chaining).

• 1 mark: explaining how the first method works and its priority behaviour.

• 1 mark: correctly naming the second method to handle multiple interrupts.

• 1 mark: explaining how the second method works and its priority behaviour.

• 2 marks: giving one correct example of a non-maskable interrupt (1 mark for a valid example, 1 mark for indicating it is non-maskable/critical).

• 2 marks: explaining why DMA is faster (1 mark for direct memory-to-device transfer without CPU per-item involvement; 1 mark for contrast with CPU overhead in programmed/interrupt-driven I/O).

• 2 marks: defining hot connection (1 mark for connect/disconnect while powered on; 1 mark for no reboot/shutdown required or hot-swappable/plug-and-play).

Model confidence: high.

Model note: The question text contains '1/0' which is assumed to be 'I/O'. The answer lists two common methods; other valid methods (vectored interrupts, multiple interrupt lines) would also be acceptable. The marking guide allocates the full 10 marks as specified in the subparts.

QUESTION A6 (10 marks)

Topic: Computer architecture and CPU organisation

Source: CSC429 (20).pdf, page(s) 3

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The CPU is the part of the computer that fetches, decodes and executes instructions, and it is conventionally described in terms of the following components:

• Control Unit (CU): directs and coordinates the operation of the whole CPU and the rest of the computer. It decodes each instruction held in the Instruction Register, generates the timing and control signals that select the required functional units, and sequences the machine cycle (fetch, decode, execute, and where relevant memory read/write or interrupt handling). It does not itself process data; it tells the other units what to do and when.

• Arithmetic Logic Unit (ALU): performs the actual data processing — arithmetic operations (add, subtract, multiply, divide, increment/decrement) and logical operations (AND, OR, NOT, XOR, shift, compare). It takes its operands from registers, produces a result that is written back to a register, and sets condition/status flags (zero, carry, sign, overflow) in the status register.

• Registers: small, very fast storage inside the CPU that hold the data and addresses currently in use. Typical registers include the Program Counter (PC), which holds the address of the next instruction; the Instruction Register (IR), which holds the instruction being executed; the Memory Address Register (MAR), which holds the address to be accessed; the Memory Data/Buffer Register (MDR/MBR), which holds data read from or to be written to memory; the Accumulator (AC) or general-purpose registers for operands and results; and the Status/Flag register. Registers remove the need to go to memory for every intermediate value and so speed up execution.

© Hak Cipta Universiti Teknologi MARA

• Internal buses / data paths: the internal connections (and the system bus interface) that carry data, addresses and control signals between the CU, ALU, registers and memory, allowing operands and results to move between components.

• Clock: generates the regular pulses that synchronise the steps of the machine cycle; the CU uses it to time each micro-operation.

(b) The bus system (address bus, data bus and control bus) is the only path over which the CPU can exchange information with memory and I/O. If it is faulty, the machine cycle breaks down at the fetch stage: the PC supplies the address, but a faulty address bus means the wrong location (or no location) is selected; a faulty data bus means the instruction word arriving in the IR is corrupted or absent; and a faulty control bus means the memory-read/write or timing signals are not delivered, so memory does not respond. The CU therefore cannot obtain a valid instruction to decode, so decoding and execution cannot proceed correctly — the CPU may hang, fetch garbage and execute invalid operations, or repeatedly fail/retry the fetch. In short, the machine cycle cannot complete normally because every stage depends on the bus to move instructions, operands and results between the CPU and memory.

Unofficial marking guide:

• 1 mark: identifies the Control Unit and states that it coordinates/sequences operations and decodes instructions and issues control signals

• 1 mark: identifies the ALU and states that it performs arithmetic and logical operations

• 1 mark: identifies registers and gives their function (fast internal storage for data/addresses/instructions)

• 1 mark: names at least two specific registers (e.g. PC, IR, MAR, MDR, accumulator) with correct roles

• 1 mark: identifies internal buses/data paths and their role in moving data/addresses/control signals between components

• 1 mark: identifies the clock and its role in synchronising/timing the machine cycle

• 1 mark: explains that the components work together under CU control to fetch, decode and execute instructions (overall CPU function)

• 1 mark: accurate, well-organised discussion covering all main components (quality/coverage)

• 1 mark: (b) explains that a faulty bus prevents correct fetching of instructions/operands (address, data or control signals corrupted or not delivered)

• 1 mark: (b) explains the consequence — the machine cycle cannot complete/decode/execute correctly, so the CPU hangs, executes garbage or fails

Model confidence: high.

Model note: Question text has a stray apostrophe in (a); treated as a typo. The mark split (8 + 2) is taken from the subpart labels. Answer assumes the standard von Neumann CPU component set (CU, ALU, registers, internal buses, clock); if the course lists a different fixed set of components, map the points accordingly.

QUESTION A7 (10 marks)

Topic: Memory hierarchy and cache

Source: CSC429 (7).pdf, page(s) 4

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) A replacement algorithm is needed in a cache because the cache is much smaller than main memory, so when a cache miss occurs and the required block must be brought into the cache, the cache may already be full. The replacement algorithm decides which existing cache block (line) to evict to make room for the new block. A good replacement policy aims to minimise future misses by evicting the block least likely to be used again, thereby improving hit ratio and overall performance.

(b) Common cache replacement policies include:

1. Random replacement: a block is chosen at random for eviction. It is simple to implement in hardware and requires no bookkeeping, but it may evict a frequently used block and gives poorer hit ratios than informed policies.
2. FIFO (First-In, First-Out): the block that has been in the cache the longest is evicted. It is easy to implement using a queue or counter, but it ignores usage frequency and can suffer from Belady's anomaly (more frames causing more misses).
3. LRU (Least Recently Used): the block that has not been accessed for the longest time is evicted, based on temporal locality. It generally gives good hit ratios but requires tracking of access order (e.g., counters or a stack), which is costly in hardware for high associativity.
4. LFU (Least Frequently Used): the block with the smallest access count is evicted. It favours frequently used blocks but requires a counter per block and can retain stale blocks that were heavily used in the past.
5. Optimal (MIN/OPT): the block that will not be used for the longest time in the future is evicted. It gives

© Hak Cipta Universiti Teknologi MARA

the best possible hit ratio but is unrealisable because it requires future knowledge; it is used only as a benchmark.

6. Other variants: NRU (Not Recently Used) uses a reference bit and is a coarse approximation of LRU; second-chance (clock) gives blocks a second chance based on a reference bit; pseudo-LRU approximates LRU with less hardware overhead.

Unofficial marking guide:

• 1 mark: explains cache is smaller than main memory and replacement is needed when a new block must be loaded into a full cache

• 1 mark: explains purpose is to choose a victim block to minimise future misses / improve hit ratio

• 2 marks: describes random replacement (selection and simple implementation)

• 2 marks: describes FIFO replacement (oldest block evicted, queue/counter implementation)

• 2 marks: describes LRU replacement (least recently used evicted, temporal locality, hardware cost)

• 1 mark: describes LFU replacement (least frequently used evicted, counter per block)

• 1 mark: describes optimal replacement (future knowledge, benchmark only)

• 1 mark: mentions at least one approximation/variant such as NRU, second-chance/clock, or pseudo-LRU

Model confidence: high.

Model note: The question asks to 'discuss all different types'; the answer lists the main policies and common approximations. Some textbooks may classify policies differently; the marking guide accepts any reasonable set of policies with correct descriptions. No figure or table is required.

QUESTION A8 (10 marks)

Topic: Input/output and interrupts

Source: CSC429 (18).pdf, page(s) 4

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Comparison of two system bus timing approaches [6 marks]

Synchronous bus timing: All bus transactions are governed by a fixed, common clock. Every device on the bus derives its timing from the same clock signal, and events (address assertion, data transfer, acknowledgement) occur at predetermined clock edges. Because the timing is fixed, the bus protocol is simple and fast, and little control logic is needed. Example: the classic synchronous buses such as the processor-memory bus in simple systems, or a synchronous DRAM (SDRAM) interface where read/write operations are clocked. The main drawback is that the bus must be designed for the slowest device, so fast devices are held back and slow devices limit overall performance.

Asynchronous bus timing: There is no common clock; instead, a handshaking protocol (e.g. request/acknowledge or strobe/acknowledge lines) coordinates the sender and receiver. The master asserts a request (or address strobe) and the slave returns an acknowledge when it is ready, so each transfer takes as long as the devices actually need. Example: the asynchronous handshake used on many I/O and peripheral buses, and the request/acknowledge handshake between a CPU and a slow memory or I/O device. This accommodates devices of widely differing speeds and is flexible, but it requires more control lines and handshake logic, making it slower per transaction and more complex than a synchronous bus.

Key comparison: synchronous = fixed clock, simple/fast but limited by slowest device; asynchronous = handshake, flexible and speed-adaptive but more complex and with extra overhead.

(b) One reason why an I/O module is needed [2 marks]

An I/O module is needed to act as the interface (translator) between the system bus and external devices, because external devices differ greatly from the CPU in data format, speed, and control signalling. The module performs functions such as data buffering, format/signal conversion, error detection, and control/timing, and it relieves the CPU of low-level device management (e.g. via interrupts or DMA). Without it, the CPU would have to deal directly with the varied electrical and timing characteristics of each peripheral.

(c) Two interfaces to external devices [2 marks]

Any two of: parallel interface (e.g. parallel port / Centronics printer port); serial interface (e.g. RS-232 / UART); USB (Universal Serial Bus); SCSI/SAS; FireWire (IEEE 1394); Ethernet (network interface); SATA; HDMI/display interface; PCIe (as an expansion/peripheral interface).

Unofficial marking guide:

• 1 mark: correctly identifies synchronous bus timing and explains fixed common clock

• 1 mark: gives an appropriate example of synchronous timing (e.g. SDRAM/processor-memory bus)

© Hak Cipta Universiti Teknologi MARA

• 1 mark: correctly identifies asynchronous bus timing and explains handshaking (request/acknowledge)
• 1 mark: gives an appropriate example of asynchronous timing (e.g. CPU-to-slow-I/O handshake)
• 1 mark: states a valid advantage/limitation of synchronous timing (simple/fast but limited by slowest device)
• 1 mark: states a valid advantage/limitation of asynchronous timing (flexible/speed-adaptive but more complex/overhead)
• 1 mark: states a valid reason an I/O module is needed (interface/translation between bus and device)
• 1 mark: explains the reason (buffering, format/signal conversion, timing/control, relieving CPU)
• 1 mark: lists one correct external-device interface (e.g. USB, serial, parallel)
• 1 mark: lists a second correct external-device interface
Model confidence: high.
Model note: Question text is clear; no OCR correction needed. Part (c) accepts any two standard external-device interfaces. This is an unofficial model answer, not an official marking scheme.

QUESTION A9 (10 marks)

Topic: Computer architecture and CPU organisation

Source: CSC429 (3).pdf, page(s) 3

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Three types of system interconnection [6 marks]

1. Bus interconnection: A set of shared parallel conducting lines (address bus, data bus and control bus) that connects the CPU, main memory and I/O devices. Only one transfer can take place on the bus at a time, so it is simple and low-cost but becomes a bottleneck as more devices are added; performance is improved by using separate or multiple buses and wider data lines.

2. Point-to-point interconnection: A dedicated link (or set of links) directly connecting two components, e.g. CPU-to-memory or CPU-to-I/O controller. Because the link is not shared, it gives higher bandwidth and avoids arbitration contention; it is used in modern systems such as QuickPath Interconnect (QPI) and PCI Express, where switches build a network of point-to-point links.

3. Interconnection network (switched/multistage network): A network of switches and links that routes messages between many components, e.g. a crossbar switch or multistage interconnection network (omega network) in multiprocessors. It allows several simultaneous transfers and is scalable, but costs more and needs routing/arbitration logic.

(Other acceptable types: shared memory interconnection, I/O channel, cluster/communication-line interconnection.)

(b) Two sub-components of the CPU and their functions [4 marks]

1. Control Unit (CU): Fetches and decodes instructions and generates the timing and control signals that direct the ALU, registers and memory/I/O to carry out each instruction (the 'conductor' of the CPU).

2. Arithmetic Logic Unit (ALU): Performs arithmetic operations (add, subtract, multiply, divide) and logical operations (AND, OR, NOT, XOR, shift, compare) on data held in registers.

(Other acceptable sub-components: registers such as PC, IR, MAR, MBR, general-purpose registers, and cache, each with its stated function.)

Unofficial marking guide:
• 1 mark: correct naming of first type of system interconnection
• 1 mark: correct explanation/description of first type
• 1 mark: correct naming of second type of system interconnection
• 1 mark: correct explanation/description of second type
• 1 mark: correct naming of third type of system interconnection
• 1 mark: correct explanation/description of third type
• 1 mark: correct naming of first CPU sub-component
• 1 mark: correct function of first CPU sub-component
• 1 mark: correct naming of second CPU sub-component
• 1 mark: correct function of second CPU sub-component
Model confidence: high.
Model note: Figure 1 is only available as extracted text; the answer relies on standard computer-organization knowledge rather than reading the figure. The question text does not specify which three interconnection types the lecturer intended, so bus, point-to-point and interconnection-network are given as the most common; any three correctly explained types should be accepted. This is an unofficial model answer, not a marking scheme.

© Hak Cipta Universiti Teknologi MARA

QUESTION A10 **(10 marks)**

Topic: Data representation and number systems

a) Answer: left -24 -> 1 0011000; right 63 -> 0 0111111; -24 - 63 = -87 -> sign-magnitude 1 1010111

- *1 mark for operand representations.*
- *1 mark for sign handling.*
- *1 mark for magnitude arithmetic.*
- *1 mark for the result sign bit.*
- *1 mark for the final byte.*

b) Answer: left -24 -> 1's 11100111, 2's 11101000; right 63 -> 1's 00111111, 2's 00111111; -24 - 63 = -87 -> 1's 10101000, 2's 10101001

- *1 mark for the positive operand in binary.*
- *1 mark for the 1's complement of the negative operand.*
- *1 mark for adding 1 to form the 2's complement.*
- *1 mark for the correct 8-bit result.*
- *1 mark for converting the result back to decimal.*

© Hak Cipta Universiti Teknologi MARA

PART B — MARKING GUIDE

QUESTION B1 (20 marks)

Topic: Assembly-language programming

Source: CSC429 (21).pdf, page(s) 6

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

Assumption on the equation (OCR correction): Figure 5 reads z-A+c2/B*D-3A. The extracted text is garbled; the standard form of this classic question is taken to be z = A + (C × 2) / (B × D) − 3A, i.e. z = A + C*2/B*D - 3A. If the figure actually shows a different arrangement (e.g. C^2 or C*2), the same instruction sequences apply with the operands reordered; the structure of the answer is unchanged. Registers A, B, C, D are fixed; x, y, z are scratch; result must end in z.

Zero-address (stack) program. All operands are pushed onto the stack; operations pop their operands and push the result. Evaluate left-to-right with the usual precedence (× and / before + and −):

```
PUSH A        ; stack: A
PUSH C
PUSH 2
MUL           ; C*2
PUSH B
PUSH D
MUL           ; B*D
DIV           ; (C*2)/(B*D)
ADD           ; A + (C*2)/(B*D)
PUSH 3
PUSH A
MUL           ; 3A
SUB           ; A + (C*2)/(B*D) - 3A
POP z         ; store final answer in z
```

One-address program. One operand is implicit in the accumulator; the named operand is the memory/register operand. Using x and y as temporaries:

```
LOAD C
MUL 2         ; ACC = C*2
STOR x        ; x = C*2
LOAD B
MUL D         ; ACC = B*D
STOR y        ; y = B*D
LOAD x
DIV y         ; ACC = (C*2)/(B*D)
ADD A         ; ACC = A + (C*2)/(B*D)
STOR x        ; save partial result in x
LOAD 3
MUL A         ; ACC = 3A
STOR y        ; y = 3A
LOAD x
SUB y         ; ACC = A + (C*2)/(B*D) - 3A
STOR z        ; final answer in z
```

Three-address program. Each instruction is OP (dest, src1, src2); temporaries x and y hold intermediate results:

```
MUL (x, C, 2)      ; x = C*2
MUL (y, B, D)      ; y = B*D
DIV (x, x, y)      ; x = (C*2)/(B*D)
ADD (x, A, x)      ; x = A + (C*2)/(B*D)
MUL (y, 3, A)      ; y = 3A
SUB (z, x, y)      ; z = A + (C*2)/(B*D) - 3A
```

In all three versions the final value is placed in register z, and the fixed registers A, B, C, D are never overwritten (only x and y are used as scratch).

© Hak Cipta Universiti Teknologi MARA

Unofficial marking guide:

• 4 marks: correct zero-address program — correct PUSH/POP ordering, correct use of MUL/DIV/ADD/SUB, result POPped into z

• 4 marks: correct one-address program — correct LOAD/STOR/MUL/DIV/ADD/SUB with accumulator, temporaries x and y used correctly, result STORed to z

• 4 marks: correct three-address program — correct three-operand MUL/DIV/ADD/SUB form, temporaries used, result written to z

• 4 marks: correct evaluation order/precedence (× and ÷ before + and −), correct handling of the 3A term, and no corruption of fixed registers A, B, C, D

• 4 marks: overall correctness and consistency across the three programs (same computed expression, final answer in z in every case)

Model confidence: medium.

*Model note: The equation in Figure 5 is OCR-garbled ('z-A+c2/B*D-3A'); I conservatively interpreted it as z = A + C*2/(B*D) − 3A, the standard form of this question. If the figure shows C² or a different grouping, the operand order changes but the instruction structure and marking allocation remain valid. The exact register/operand syntax of the three-address form is taken from Table 1 as OP(p,q,r).*

END OF MARKING SCHEME

© Hak Cipta Universiti Teknologi MARA